



Green University of Bangladesh
Department of Computer Science and Engineering (CSE)
Faculty of Sciences and Engineering
Semester: (Fall, Year:2026), B.Sc. in CSE (Day)

Lab Report : 01

Course Title: Design Project I

Course Code: CSE 308

Section: 241-D2

Title: AI-powered personal knowledge and research engine that transforms documents and information into a connected knowledge graph.

Student Details

	Name	ID
1.	Ziham Mahmud	241002007
2.	MD. Abdullah Al Mamun Mullah	241002033
3.	Faria Khan Purnata	241002038

Submission Date : 06/10/2026

Course Teacher's Name : Mr. Montaser Abdul Quader

[For Teachers use only: **Don't Write Anything inside this box**]

Project Proposal Status	
Marks: _____	Signature: _____
Comments: _____	Date: _____

Contents

1	Introduction	3
2	Problem Statement	4
3	Motivation	5
4	Objectives	6
5	System Architecture	7
5.1	Architectural Components	7
6	Methodology	9
6.1	Phase 1: Multi-Format Document Ingestion & Preprocessing	9
6.2	Phase 2: Entity, Concept, and Relationship Extraction	9
6.3	Phase 3: Entity Resolution and Graph Construction	9
6.4	Phase 4: Hybrid GraphRAG Querying and Visualization	10
7	Mathematical Graph Formulation	11
7.1	Formal Definition of Knowledge Graph	11
7.2	Semantic Triples Formulation	11
7.3	Matrix and Tensor Representation	12
7.4	Path Formulation and Multi-Hop Traversal	12
7.5	Hybrid Graph-Vector Semantic Retrieval	12
8	Proposed Technology Stack	13
9	Implementation Roadmap	14
10	Work Breakdown Structure (WBS)	15
10.1	Task Decomposition	15
11	Evaluation Strategy	17
11.1	Evaluation Metrics and Objectives	17
11.2	Testing and Validation Methodologies	17
12	Conclusion	19

1 Introduction

In the modern digital era, students, researchers, developers, and professionals deal with a large amount of information distributed across different sources. Research papers, PDF documents, personal notes, web content, source code, and other digital resources are often stored separately. Traditional document management systems mainly treat these resources as isolated files and provide limited keyword-based search capabilities.

This makes it difficult for users to understand how different pieces of information are connected. A researcher may know that relevant information exists in several documents but may still have to manually read and compare them to identify relationships between concepts.

To address this problem, this project proposes **Nexora**, an AI-powered personal knowledge and research engine. Nexora transforms scattered information into a connected and searchable knowledge environment.

The system uses Artificial Intelligence (AI), Natural Language Processing (NLP), semantic search, and knowledge graph technologies to extract entities, concepts, relationships, and source references from different types of information. The extracted knowledge is represented as a dynamic knowledge graph that users can explore and query using natural language.

The primary goal of Nexora is not simply to retrieve information but to help users understand the relationships among different pieces of information.

2 Problem Statement

The exponential growth of digital information has created a significant challenge for students, researchers, and professionals in managing and utilizing knowledge effectively. Information is currently fragmented across disparate formats—including PDF documents, research papers, personal notes, web pages, and source-code repositories—and stored in isolated file systems.

Existing conventional document management and retrieval systems suffer from several core limitations:

- **Isolated Storage:** Documents exist as standalone files without inherent connectivity to related information.
- **Keyword-Based Retrieval:** Search mechanisms rely heavily on exact string matching rather than understanding contextual meaning or underlying semantics.
- **Hidden Interrelationships:** Logical connections, shared concepts, and cross-document dependencies remain unmapped, requiring users to manually read and compare multiple sources.
- **High Cognitive Load and Inefficiency:** Extracting insights, synthesizing information across multiple documents, and maintaining source attribution is time-consuming and prone to human error.
- **Lack of Visualization:** Complex research themes and topic hierarchies cannot be easily visualized or navigated interactively.

Consequently, there is a critical need for an intelligent personal research assistant that automates the process of information extraction, semantic relationship mapping, and dynamic knowledge graph construction. **Nexora** directly addresses this gap by transforming static, unstructured content into an interconnected, searchable, and visual knowledge ecosystem.

3 Motivation

The primary motivation behind developing **Nexora** stems from the growing disparity between the vast volume of information users collect and their ability to effectively organize, synthesize, and extract meaningful insights from it.

During academic research and professional work, individuals frequently gather hundreds of digital documents, ranging from research papers and lecture notes to technical articles and source code. However, conventional storage mechanisms treat these files as static, isolated entities. As a result:

- **Knowledge Bottlenecks:** Valuable concepts and implicit connections remain hidden across separate files, forcing researchers to spend substantial time manually cross-referencing materials.
- **Contextual Disconnection:** Traditional file systems fail to capture how technical concepts evolve or relate to one another (e.g., how *Neural Networks* underpin *Deep Learning*, which in turn is a subfield of *Machine Learning*).
- **Cognitive Fatigue:** Reading through voluminous documents repeatedly to recall specific references or synthesize comparative findings creates unnecessary cognitive strain and reduces overall research productivity.

By leveraging modern advancements in Artificial Intelligence (AI), Natural Language Processing (NLP), and Knowledge Graph technologies, **Nexora** addresses these challenges directly. The project is driven by the vision of bridging the gap between raw unstructured data and structured semantic intelligence—transforming passive document storage into an active, connected, and navigable research partner.

4 Objectives

The primary goal of the **Nexora** project is to design, develop, and evaluate an intelligent personal knowledge and research engine that automates document analysis and builds an interactive semantic knowledge graph.

To achieve this, the specific objectives of the project are:

1. **Heterogeneous Information Ingestion:** To build an ingestion engine capable of parsing and extracting structured text from diverse document formats (e.g., PDFs, DOCX, Markdown, plain text, web content, and source code).
2. **Automated Entity and Concept Extraction:** To implement Natural Language Processing (NLP) and AI techniques to recognize key entities, technical terms, and core concepts across uploaded materials.
3. **Semantic Relationship Discovery:** To automatically identify, extract, and classify semantic relationships and dependencies linking extracted concepts across multiple documents.
4. **Knowledge Graph Construction:** To model and store extracted entities and relationships as nodes and directed edges within a dynamic graph database infrastructure (e.g., Neo4j).
5. **Contextual Semantic Search and Q&A:** To integrate vector database retrieval and Large Language Model (LLM) processing to enable natural language querying, summarization, and direct question answering.
6. **Interactive Knowledge Visualization:** To develop a user-friendly, interactive graphical interface that allows users to visually navigate, inspect, and filter nodes and connections in their knowledge graph.
7. **Source Traceability and Citation Preservation:** To maintain strict mappings between graph entities and their original source documents to ensure precise reference tracking and combat hallucination.
8. **Efficiency and Cognitive Load Reduction:** To minimize the manual overhead involved in cross-document research, literature review, and knowledge management for students, researchers, and professionals.

5 System Architecture

The system architecture of **Nexora** is designed as a modular, multi-tiered pipeline that converts unstructured user inputs into an interconnected knowledge graph and delivers context-aware, semantic answers.

The functional layout of the system consists of six major integrated layers:

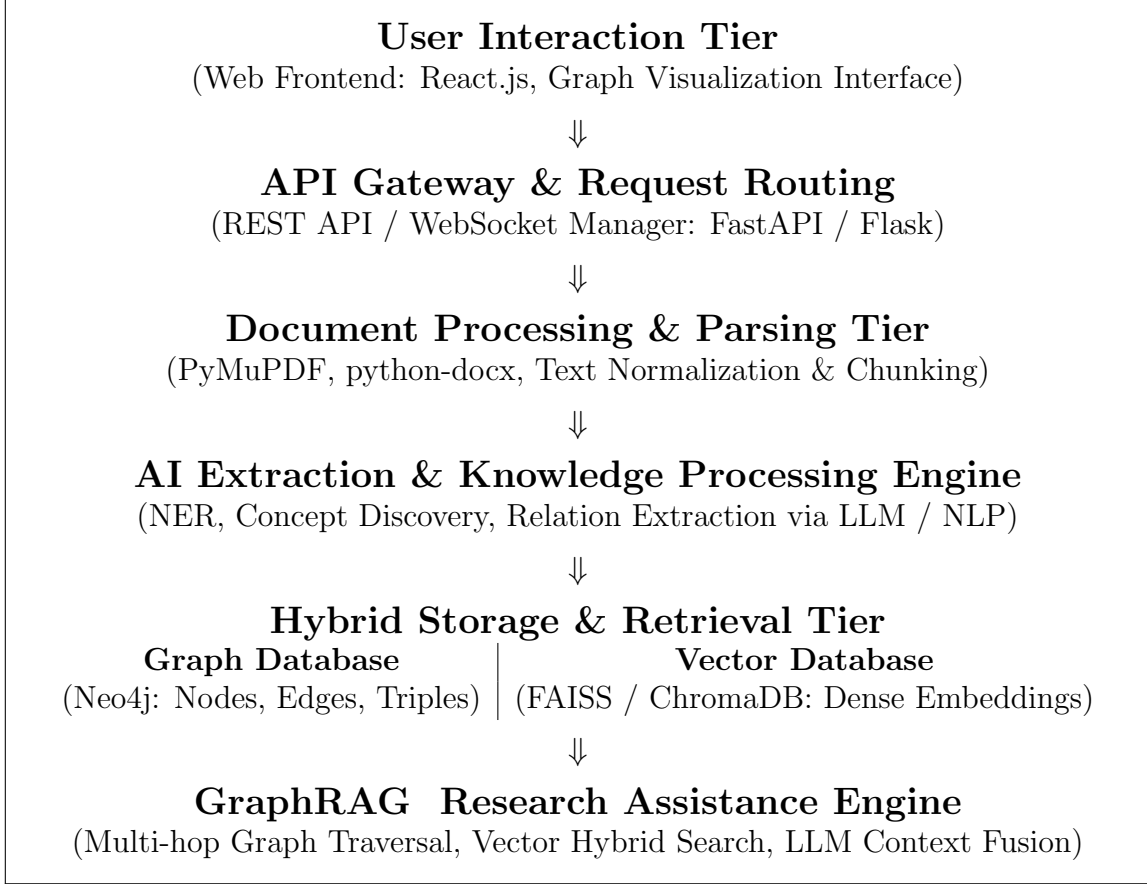


Figure 1: Layered Architectural Pipeline of Nexora

5.1 Architectural Components

- **User Interface Layer:** Provides an interactive portal where users upload multi-format documents, issue natural language queries, and interactively explore generated graph structures.
- **Ingestion & Extraction Layer:** Extracts raw text from incoming files (PDFs, DOCX, code repositories) and partitions text into semantic chunks while maintaining source document line-numbers and metadata.
- **Knowledge Graph Engine:** Processes semantic chunks through NLP and LLM pipelines to extract subject-predicate-object triples (S, P, O) and constructs schema-constrained graph representations.
- **Hybrid Storage Tier:** Utilizes **Neo4j** for graph topology, edge traversal, and multi-hop query evaluation, combined with **FAISS / ChromaDB** for dense semantic vector search.

- **GraphRAG Retrieval Engine:** Executes hybrid context retrieval by combining vector-similarity ranking with graph traversals (neighbor lookups and multi-hop paths) to formulate contextually grounded prompts for the LLM.

6 Methodology

The methodology behind **Nexora** follows an end-to-end processing pipeline that translates raw unstructured information into a dynamic, queryable knowledge environment.

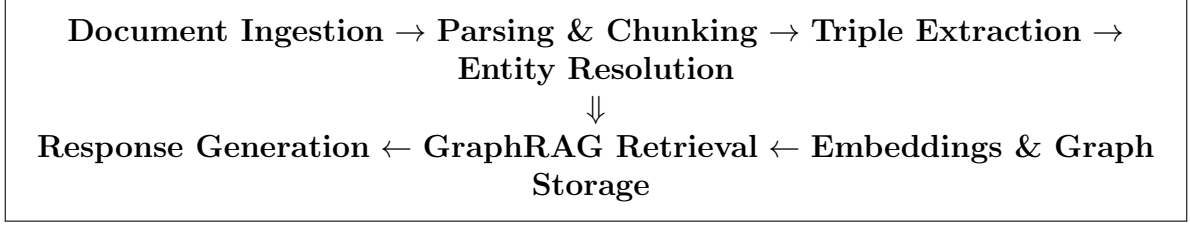


Figure 2: Step-by-Step Methodology Workflow

The step-by-step technical methodology comprises the following stages:

6.1 Phase 1: Multi-Format Document Ingestion & Preprocessing

1. **Parsing:** Incoming files (PDFs, Word documents, Markdown, plain text) are converted into standardized plain text using specialized parsers like `PyMuPDF` and `python-docx`.
2. **Text Normalization:** Extracted text undergoes cleaning (removal of noise, fix encoding issues, line-break consolidation).
3. **Semantic Chunking:** The normalized text is split into semantic chunks of uniform token length with overlapping windows to prevent loss of boundary context.

6.2 Phase 2: Entity, Concept, and Relationship Extraction

1. **Named Entity Recognition (NER) & Concept Identification:** Large Language Models (LLMs) and advanced NLP pipelines analyze each chunk to isolate key technical concepts, entities, and domain terms.
2. **Relation Extraction:** The system identifies directional dependencies between extracted entities, mapping relationships in subject-predicate-object semantic triples:

$$\text{Triple} = (\text{Entity}_{\text{subject}}, \text{Relationship}_{\text{predicate}}, \text{Entity}_{\text{object}})$$

Example: (Deep Learning) $\xrightarrow{\text{is-a-subfield-of}}$ (Machine Learning)

6.3 Phase 3: Entity Resolution and Graph Construction

1. **Entity Canonicalization:** Duplicate or synonymously referenced entities across separate documents (e.g., "ML", "Machine Learning", "Mach. Learning") are consolidated into a single unified canonical node to ensure topological integrity.
2. **Graph Population:** Canonical entities and their directed semantic relationships are written to the Neo4j graph database as nodes V and edges E :

$$G = (V, E)$$

3. **Vector Embedding Indexing:** Parallel dense vector representations of textual chunks and entity descriptions are generated using embedding models and stored in the vector database for fast semantic lookup.

6.4 Phase 4: Hybrid GraphRAG Querying and Visualization

1. **Dual-Route Retrieval:** When a user asks a natural language question:
 - *Vector Search* retrieves top- K semantically relevant chunks.
 - *Graph Traversal* extracts multi-hop paths and relational subgraphs connecting the identified concepts.
2. **Context Fusion & Response Generation:** The graph subgraphs, vector text chunks, and exact source citations are fused into a structured prompt context, allowing the LLM to generate precise, hallucination-free answers with verifiable source tracking.
3. **Interactive Visualization:** The web application renders the retrieved subgraph using libraries such as Cytoscape.js, enabling users to visually explore relationships.

7 Mathematical Graph Formulation

To formally define the structured knowledge representation used in **Nexora**, the system models extracted unstructured information as a directed, multi-relational, typed knowledge graph.

7.1 Formal Definition of Knowledge Graph

A knowledge graph is represented as a directed labeled graph tuple:

$$G = (V, E, R, L_V, L_E, \phi)$$

where:

- $V = \{v_1, v_2, \dots, v_n\}$ represents the finite set of **Entity / Concept Nodes**.
- $E \subseteq V \times R \times V$ represents the set of **Directed Semantic Edges** (Relationships).
- $R = \{r_1, r_2, \dots, r_m\}$ represents the set of distinct **Relationship Predicates**.
- $L_V : V \rightarrow \mathcal{T}_V$ is a function mapping nodes to entity categories (e.g., Topic, Algorithm, Document).
- $L_E : E \rightarrow \mathcal{T}_E$ is a function mapping edges to relationship types (e.g., is-a-subfield-of, uses, reference).
- $\phi : V \cup E \rightarrow \mathbb{R}^d$ is a feature mapping assigning a d -dimensional dense embedding vector to every node and edge.

7.2 Semantic Triples Formulation

Each connection in the graph G is formally structured as a knowledge triple:

$$\tau = (s, p, o) \in E$$

where:

- $s \in V$ is the **Subject entity** (Head node).
- $p \in R$ is the **Predicate relationship** (Edge type).
- $o \in V$ is the **Object entity** (Tail node).

Example Formulation:

$$\tau_1 = (\text{Deep Learning}, \text{is-a-subfield-of}, \text{Machine Learning})$$

$$\tau_2 = (\text{Transformer Model}, \text{utilizes}, \text{Self-Attention Mechanism})$$

7.3 Matrix and Tensor Representation

For computational processing and graph neural network operations, G is represented using an ****Adjacency Tensor**** $\mathcal{A} \in \{0, 1\}^{|V| \times |R| \times |V|}$:

$$\mathcal{A}_{i,k,j} = \begin{cases} 1, & \text{if } (v_i, r_k, v_j) \in E \\ 0, & \text{otherwise} \end{cases}$$

When considering a specific relation type r_k , the relation-specific adjacency matrix $A^{(k)} \in \mathbb{R}^{|V| \times |V|}$ is defined as:

$$A_{i,j}^{(k)} = \mathcal{A}_{i,k,j}$$

7.4 Path Formulation and Multi-Hop Traversal

To evaluate relationships across distant concepts, a **Multi-Hop Path** P of length k between source node v_0 and target node v_k is defined as an alternating sequence of nodes and edge predicates:

$$P = (v_0, r_1, v_1, r_2, v_2, \dots, r_k, v_k)$$

where $\forall i \in \{1, 2, \dots, k\}, (v_{i-1}, r_i, v_i) \in E$.

The existence of a path of length k for a sequence of relation matrices $A^{(1)}, A^{(2)}, \dots, A^{(k)}$ is calculated via matrix multiplication:

$$M_{\text{reachability}} = \prod_{m=1}^k A^{(m)}$$

If $(M_{\text{reachability}})_{i,j} > 0$, there exists at least one k -hop semantic path connecting node v_i to node v_j .

7.5 Hybrid Graph-Vector Semantic Retrieval

To combine structural topology with vector similarity during search, the total semantic relevance score $S(q, v)$ between a user query embedding $\mathbf{q} \in \mathbb{R}^d$ and a candidate concept node $v \in V$ with embedding $\mathbf{h}_v \in \mathbb{R}^d$ is computed as:

$$S(q, v) = \alpha \cdot \cos(\mathbf{q}, \mathbf{h}_v) + (1 - \alpha) \cdot \text{PageRank}_{\text{subgraph}}(v)$$

where:

- $\cos(\mathbf{q}, \mathbf{h}_v) = \frac{\mathbf{q} \cdot \mathbf{h}_v}{\|\mathbf{q}\| \|\mathbf{h}_v\|}$ measures dense semantic similarity.
- $\text{PageRank}_{\text{subgraph}}(v)$ measures topological importance within the local subgraph.
- $\alpha \in [0, 1]$ is a hyperparameter balancing vector proximity and graph centrality.

8 Proposed Technology Stack

Component	Technology
Frontend Framework	React.js, TypeScript
UI Styling	Tailwind CSS
Graph Visualization	Cytoscape.js, D3.js
Backend Framework	Python, FastAPI
Knowledge Graph Database	Neo4j
Vector Database	ChromaDB, FAISS
Relational Database	PostgreSQL
AI / NLP Framework	LangChain, LlamaIndex
Language Models	OpenAI GPT-4o
Document Processing	PyMuPDF, python-docx
API Communication	REST API, WebSockets
Containerization	Docker
Version Control	Git, GitHub

Proposed Technology Stack

Note on Technology Evolution: The selection of tools and libraries listed above reflects the initial architecture proposed for the project. As development progresses, system requirements, performance benchmarks, component compatibility, or new technical insights may necessitate changes, updates, or additions to this technology stack to ensure optimal performance and scalability.

9 Implementation Roadmap

The development of **Nexora** is structured into five sequential phases spanning a 16-week execution timeline. This phased approach ensures incremental milestone delivery, starting from document ingestion foundations to complex knowledge extraction, hybrid retrieval, and final user testing.

Table 1: Project Implementation Timeline

Phase	Core Deliverables	Duration
Phase 1: Setup & Ingestion	Project setup, document parsers (PyMuPDF, docx), text cleaning, chunking pipeline.	Weeks 1–3
Phase 2: Graph & Vector Engine	Schema definition, Neo4j setup, ChromaDB vector indexing, NER and triple (S, P, O) extraction pipelines.	Weeks 4–7
Phase 3: Hybrid Retrieval & RAG	Cypher query generation, multi-hop path traversal, GraphRAG context fusion, LLM Q&A engine.	Weeks 8–11
Phase 4: UI & Visualization	React dashboard development, Cytoscape.js interactive graph renderer, search interface integration.	Weeks 12–14
Phase 5: Evaluation & Deployment	System integration testing, retrieval accuracy benchmarks, dockerization, final report compilation.	Weeks 15–16

10 Work Breakdown Structure (WBS)

The Work Breakdown Structure details the granular tasks assigned across the system’s core functional areas:

Work Breakdown Structure (WBS)			
1. Data Ingestion	2. Graph Engine	3. RAG & Retrieval	4. Frontend UI
Multi-format Parsers Text Chunking	Entity Normalization Triple Extraction	Vector Search (FAISS) Graph Traversal	React Dashboard Cytoscape.js Visuals
Source Citation Setup	Neo4j Schema	Context Fusion	Q&A Chat Portal

Figure 3: Hierarchical Work Breakdown Overview

10.1 Task Decomposition

1. Data Ingestion & Preprocessing Module

- Configure layout-aware PDF parsing using PyMuPDF.
- Build document cleaning routines to normalize noise and text encoding.
- Implement sliding-window text chunking with metadata preservation.

2. Knowledge Graph & NLP Extraction Engine

- Design and enforce schema constraints in Neo4j for concept nodes and relational edges.
- Prompt-engineer LLMs for subject-predicate-object triple extraction.
- Implement entity resolution routines to merge duplicate nodes and synonyms.

3. Hybrid GraphRAG & Semantic Retrieval Engine

- Set up vector database indexes (ChromaDB / FAISS) for dense text chunk search.
- Write Cypher multi-hop graph traversal algorithms to collect subgraphs.
- Construct a context fusion pipeline blending vector context with graph subgraphs.

4. User Interface & Interactive Visualization

- Develop responsive dashboard components using React.js and Tailwind CSS.
- Integrate Cytoscape.js for dynamic, force-directed graph rendering.
- Build search and Q&A interfaces with clickable node inspection.

5. Integration, Benchmarking, and Evaluation

- Conduct end-to-end integration testing between FastAPI, Neo4j, and React frontend.
- Evaluate extraction precision and Q&A response grounding against benchmark data.
- Containerize application modules using Docker Compose.

11 Evaluation Strategy

To ensure that **Nexora** operates accurately, reliably, and efficiently, a comprehensive evaluation strategy will be conducted across system performance, AI accuracy, graph quality, and user satisfaction.

11.1 Evaluation Metrics and Objectives

The system will be evaluated across four core dimensions using both quantitative and qualitative metrics:

Table 2: Evaluation Dimensions, Metrics, and Target Criteria

Evaluation Dimension	Key Metrics	Target Benchmark / Goal
Information Extraction	Precision, Recall, F_1 -score	$> 85\%$ F_1 -score for NER & relation extraction
Semantic Retrieval (RAG)	Mean Reciprocal Rank (MRR), Hit@ K , Context Precision	$> 90\%$ relevance in Top-5 retrieved chunks
Response Fidelity	Faithfulness, Answer Relevance, Hallucination Rate	Zero hallucination against retrieved source documents
System Performance	Ingestion speed, Query latency, Graph render time	< 2 seconds per Q&A response; smooth graph interaction

11.2 Testing and Validation Methodologies

- (a) **Ground-Truth Benchmark Dataset Construction:** A manually curated dataset consisting of sample research papers, documents, and technical notes with pre-annotated entities, triples, and target answers will serve as the benchmark ground truth.
- (b) **Knowledge Extraction Evaluation:** Named Entity Recognition (NER) and relation extraction outputs will be compared against human annotations using standard statistical metrics:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- (c) **Hybrid GraphRAG & Q&A Validation:** The system’s retrieval precision and answer quality will be evaluated using automated RAG evaluation frameworks for faithfulness and source traceability.
- (d) **Performance and Latency Profiling:** System response times will be measured across document parsing, graph write operations, vector search latency, and frontend graph rendering.

- (e) **Usability and User Study:** Qualitative evaluation will be conducted with target users to rate ease of interface navigation and overall effectiveness in simplifying document synthesis.

12 Conclusion

In this project proposal, we presented **Nexora**, an AI-powered personal knowledge and research engine engineered to bridge the gap between fragmented, unstructured documents and structured, interconnected intelligence. Traditional document management and keyword-based retrieval systems treat resources as isolated files, forcing users to manually read, correlate, and synthesize information across multiple sources. **Nexora** solves this limitation by introducing a unified research ecosystem powered by Natural Language Processing (NLP), dynamic Knowledge Graphs (Neo4j), vector database semantic search (FAISS/ChromaDB), and Large Language Models (LLMs).

Through an end-to-end processing pipeline—encompassing multi-format document ingestion, automated Subject-Predicate-Object (S, P, O) triple extraction, entity resolution, and hybrid GraphRAG context fusion—the system transforms raw text into an interactive graph topology. Users can perform multi-hop concept traversals, conduct natural language querying, and visualize complex academic themes while preserving complete source traceability down to specific pages and citations.

The successful implementation of **Nexora** will significantly reduce cognitive fatigue and manual research overhead for students, academics, and domain professionals. By shifting the paradigm from static document searching to dynamic, relational knowledge discovery, **Nexora** provides a robust, scalable foundation for the next generation of intelligent personal research assistants.